National Institute of Technology Goa
Department of Computer Science and Engineering

# Countermeasures and Defence Mechanisms for Drone Swarm Threats

Major Project (CS499) — Mid-Semester Evaluation

---

**Group III**

Mrutyunjay Kumar Rao (22CSE1019)
Nischay Khobragade (22CSE1023)

**Guide:** Dr. Pravati Swain

13 March 2026

# Overview & Structure

1. **Phase 1 Recap & Limitations**
   Where Phase 1 ended and why it was not enough.
2. **Communication Graph & Fiedler Value**
   Modeling the swarm as a graph; algebraic connectivity $\lambda_2$.
3. **System Pipeline & DBSCAN Deployment**
   Complete 4-phase architecture and intelligent jammer initialization.
4. **Agent Design — Observation, Actor & Critic**
   5D state vector, neural network architectures, and CTDE.
5. **Reward Function & TD Learning**
   Multi-objective reward and temporal difference error.

# I : Phase 1 Recap & Limitations

# Where Phase 1 Ended — and Why It Was Not Enough

## What Phase 1 Achieved

| Component | Phase 1 Approach |
|---|---|
| Algorithm | Q-Learning + Tile Coding |
| Reward | Max received jamming power |
| Scenarios | 2v3, 4v3, 6v3 |

## Critical Limitations

- Wrong objective — maximizing power at individual drones does not measure swarm coordination loss.
- No graph metric — no way to know if the swarm is still connected as a network.
- Q-table cannot scale — state space explodes beyond 6 agents.
- Single frequency — only 2.4 GHz; real swarms use multiple bands.

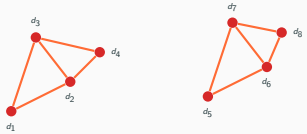
## Core Gap

High jamming power at individual drones $\neq$ swarm fragmentation

**Phase 2 objective:** prove and optimize the exact mathematical condition for swarm disconnection.

# II : Communication Graph & Algebraic Connectivity

# Enemy Swarm $\rightarrow$ Communication Graph $G = (V, E)$




$V = \{d_1, d_2, \ldots, d_n\}$ — drones are nodes
$E = \{(i, j)\}$ — active radio links are edges

**Adjacency Matrix** (5×5 subset):

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

## Edge Condition — FSPL Model

$$P_R(i, j) = P_{tx} \cdot \left( \frac{c}{4\pi f \cdot d_{ij}} \right)^2$$

Edge exists if:

- $P_R(i, j) \geq P_{\text{sens}} = -90$ dBm
- AND link is not actively jammed

$d_{ij} = \|\mathbf{x}_i - \mathbf{x}_j\|_2$

## Graph Laplacian

$$L = D - A$$

- $D$ = diagonal degree matrix
- $D[i, i]$ = number of links drone $i$ has
- $L$ encodes the complete connectivity structure of the swarm

*Recomputed every timestep as jammers move and links break.*

# Algebraic Connectivity $\lambda_2$ — One Number That Measures Swarm Coordination

## Eigen decomposition of the Laplacian $L$

$$L \cdot \mathbf{v} = \lambda \cdot \mathbf{v} \qquad \text{Eigenvalues: } 0 = \lambda_1 \leq \lambda_2 \leq \lambda_3 \leq \cdots \leq \lambda_n$$

$$\lambda_2 = \textbf{Fiedler Value} = \textbf{Algebraic Connectivity}$$

### Dense Graph

$\lambda_2 = 0.82$

Swarm fully coordinated

Dangerous

### Sparse Graph

$\lambda_2 = 0.21$

Partially disrupted

**Degraded**

### Fragmented

$\lambda_2 = 0.00$

Swarm fragmented

**Mission Complete**

**Proposition 1 — Fiedler (1973):** $\lambda_2(G) = 0 \iff G$ is disconnected ;
$\lambda_2(G) > 0 \iff G$ is connected

When our system drives $\lambda_2$ to zero, swarm fragmentation is provably guaranteed.

# III : System Pipeline & DBSCAN Deployment

# Complete System Pipeline — 4 Phases

**PHASE 1 — Environment Setup**
Spawn $N$ enemy drones · Build graph $G$ · Compute initial $\lambda_2(0)$

↓

**PHASE 2 — Clustering & Deployment**
Run DBSCAN on enemy positions · Identify $k$ centroids · Deploy $M$ jammers

↓

**PHASE 3 — MARL-PPO Training Loop**
5D observation → Actor outputs velocity + band → $\lambda_2$ recomputed → PPO update

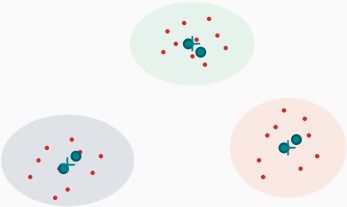
↓

**PHASE 4 — Deployment**
Load trained Actor weights · Deterministic inference · Monitor $\lambda_2$

## System Parameters

| Parameter | Value |
|---|---|
| $N$ (enemies) | 30 |
| $M$ (jammers) | 6 |
| Arena | $100\,\text{m} \times 100\,\text{m}$ |
| Episode length | 100 steps |
| Training | 1500 episodes |
| Success | $\lambda_2 < 0.2 \cdot \lambda_2(0)$ |

# DBSCAN — Intelligent Jammer Initialization



$\varepsilon = 30\,\mathrm{m}$ (neighborhood radius)
$\mathrm{minPts} = 2$ (minimum cluster size)
$k = 3$ clusters detected
$M = 6$ jammers $\rightarrow$ 2 per cluster

## Why DBSCAN?

- Enemy swarms naturally form spatial clusters.
- Random jammer placement wastes the first 100–200 steps just repositioning.
- DBSCAN finds cluster structure from enemy $(x, y)$ positions — **no labels needed**.
- Jammers start at centroids $\rightarrow$ immediately in effective range.

## DBSCAN Algorithm — 3 Steps

1. For each drone, find all drones within $\varepsilon = 30\,\mathrm{m}$.
2. If $\geq$ minPts$= 2$ neighbors $\rightarrow$ core point $\rightarrow$ form cluster.
3. Compute centroid $\mu_k$ of each cluster $k$.

Place jammers at $\mu_k \pm$ small random offset.
Re-run every 10 timesteps as swarm moves.

# IV : Agent Design — Observation, Actor & Critic

# What Each Jammer Sees — 5D State Observation

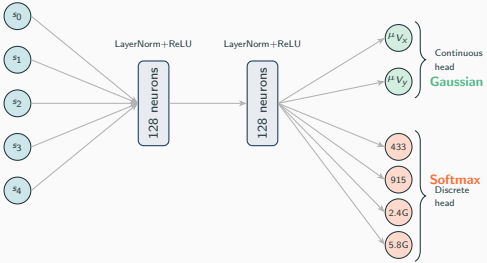
| Index | Feature | Formula | Range | Meaning |
|---|---|---|---|---|
| $s[0]$ | dist_to_centroid | $\lVert \mathbf{p}_j - \mu_k \rVert$ / arena | $[0, 1]$ | How far am I from my assigned cluster center? |
| $s[1]$ | cluster_density | $n_k$ / $N$ | $[0, 1]$ | What fraction of enemies are in my cluster? |
| $s[2]$ | dist_to_others | $\overline{\lVert \mathbf{p}_j - \mathbf{p}_{j'} \rVert}$ / arena | $[0, 1]$ | How spread out am I from teammate jammers? |
| $s[3]$ | coverage_overlap | pairs within $2R_{\text{jam}}$ / total | $[0, 1]$ | Am I wasting coverage by overlapping a teammate? |
| $s[4]$ | band_match | $\mathbf{1}[b_j = b_{\text{enemy}}]$ | $\{0, 1\}$ | Is my frequency matching the enemy band? |

## Normalization

All values clipped to [0, 1] before entering the neural network: $\mathbf{s}_{\text{norm}} = \text{clip}(\mathbf{s},\ 0,\ 1)$

Prevents any single feature from dominating the network activations.

# Actor Network — Mapping 5D Observation to Actions




## Layer Sizes

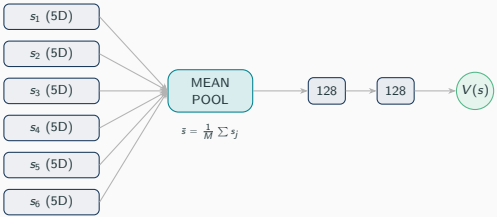
$5 \to 128$: each neuron learns a specific combination of the 5 input features.

$128 \to 128$: second layer combines simple patterns into complex jamming strategies.

**Total:** $(5 \times 128) + (128 \times 128) + (128 \times 6) \approx 18{,}000$ trainable weights.

**Parameter sharing:** all $M = 6$ jammers use ONE Actor network. Each jammer passes its own local 5D observation. Different inputs $\to$ different actions $\to$ coordinated behaviour.

# Critic Network — Estimating Team Value $V(s)$




## What Does $V(s)$ Mean?

$V(s) = 0.74$ means:
*"From this team configuration, I expect 0.74 total reward over the remaining episode."*

High $V(s) \rightarrow$ jammers well positioned, correct bands, $\lambda_2$ already falling.

Low $V(s) \rightarrow$ jammers misplaced, wrong frequency, $\lambda_2$ still high.

## Why Mean Pooling?

Critic must see the whole team, but input size must stay fixed. Mean pool collapses $M \times 5 \rightarrow 5$ — scales to any number of jammers without retraining.

**Centralized Training / Decentralized Execution (CTDE)**

**Training:** Critic uses pooled global state $\rightarrow$ better gradient estimates for Actor.
**Deployment:** Critic is completely discarded $\rightarrow$ each jammer runs Actor alone using only its own 5D observation.

# V : Reward Function & TD Learning

# Reward Function — 5 Terms Teaching the Agent What "Good" Means

| Term | Formula | Weight | Dir. | Purpose |
|---|---|---|---|---|
| $\lambda_2$ Reduction | $1 - \lambda_2(t) \,/\, \lambda_2(0)$ | $\omega_1 = 1.0$ | $\uparrow$ | **PRIMARY** — swarm fragmentation |
| Band Match | $\frac{1}{M} \sum \mathbf{1}[b_j = b_{\text{enemy}}]$ | $\omega_2 = 0.3$ | $\uparrow$ | Correct frequency selection |
| Cluster Proximity | $\frac{1}{M} \sum \exp(-\kappa \cdot d_{\text{centroid}})$ | $\omega_3 = 0.2$ | $\uparrow$ | Stay near assigned cluster |
| Energy Penalty | $\frac{1}{M} \sum \lVert \mathbf{v}_j \rVert^2 / v_{\max}^2$ | $\omega_4 = 0.1$ | $\downarrow$ | Discourage unnecessary movement |
| Overlap Penalty | overlapping pairs / total pairs | $\omega_5 = 0.2$ | $\downarrow$ | Spread coverage across clusters |

## Total Reward

$$R(t) = \omega_1 \cdot (\lambda_2 \text{ reduction}) + \omega_2 \cdot (\text{band match}) + \omega_3 \cdot (\text{proximity}) - \omega_4 \cdot (\text{energy}) - \omega_5 \cdot (\text{overlap})$$

# Temporal Difference Error — How the Agent Measures Surprise

## Core Idea

The Critic predicts expected future reward at state $s_t$. After one step, we see what actually happened.

TD error = gap between prediction and reality.

**Step 1 — Critic prediction at time $t$:**

$V(s_t)$ = "From this state, I expect $X$ total future reward"

**Step 2 — Better estimate after one step:**

$$G_t \approx r_t + \gamma \cdot V(s_{t+1})$$

$r_t$ = actual reward; $\gamma = 0.99$

**Step 3 — TD Error:**

$$\boxed{\delta_t = r_t + \gamma \cdot V(s_{t+1}) - V(s_t)}$$

**$\delta_t > 0$ — Better than expected**

Action was good → increase its probability.

**$\delta_t = 0$ — Matched prediction**

Action was average → no change needed.

**$\delta_t < 0$ — Worse than expected**

Action was bad → decrease its probability.

$\delta_t$ is computed for **every timestep** in the buffer after each rollout. It is the raw signal from which Advantage and GAE are both derived. Without $\delta_t$ there is no learning signal — it is the engine of all RL.

# From TD Error to GAE — Computing How Good Each Action Truly Was

## Why Not Use $\delta_t$ Directly?

$\delta_t$ is only one step of information.
It fully trusts $V(s_{t+1})$, which is estimated by the Critic.

Early in training $\rightarrow$ Critic is wrong $\rightarrow$ $\delta_t$ is wrong $\rightarrow$ bad learning signal.

We need to **look further ahead** for a more accurate signal.

## Advantage $A_t$

$A_t$ = "How much better was THIS action compared to average?"

$A_t > 0 \rightarrow$ above average $\rightarrow$ reinforce
$A_t < 0 \rightarrow$ below average $\rightarrow$ suppress

True: $A_t = Q(s_t, a_t) - V(s_t)$
But $Q$ is unknown $\rightarrow$ we must **estimate** it.

## Generalised Advantage Estimation (GAE)

1-step: $\hat{A}_t^{(1)} = \delta_t$

2-step: $\hat{A}_t^{(2)} = \delta_t + \gamma\delta_{t+1}$

$n$-step: $\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l \delta_{t+l}$

**GAE** — exponentially weighted sum of all $n$-step estimates:

$$\boxed{A_t^{\text{GAE}} = \delta_t + (\gamma\lambda)\, A_{t+1}^{\text{GAE}}}$$

Computed backwards from $t = T$ to $t = 0$.

**Example — last 4 steps:** $\delta_{47} = -0.02$, $\delta_{48} = +0.15$, $\delta_{49} = +0.41$, $\delta_{50} = 0$

$A_{50} = 0 \quad A_{49} = 0.41 \quad A_{48} = 0.15 + 0.94 \times 0.41 = +\mathbf{0.535} \quad A_{47} = -0.02 + 0.94 \times 0.535 = +\mathbf{0.483}$

Step 47 looks bad in isolation ($\delta = -0.02$), but GAE shows it was actually good ($+0.483$) because it set up the big reward at step 49.

# VI : PPO — Proximal Policy Optimization

# PPO — Why Standard Policy Gradient Breaks

## Standard Policy Gradient

$$\theta \leftarrow \theta + \alpha A_t \nabla_\theta \log \pi_\theta(a_t|s_t)$$

Large $A_t \Rightarrow$ very large update $\Rightarrow$ policy changes drastically.

**Issue:** No control over how much policy changes per update.

## PPO Fix — Probability Ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

$r_t \in [0.8, 1.2]$ ensures policy changes $\leq 20\%$ per update

## Four Cases

| $A_t$ | $r_t$ | PPO Behaviour | Result |
|---|---|---|---|
| + large | 1.8 | clipped to 1.2 | capped |
| + small | 1.1 | not clipped | allowed |
| − large | 0.3 | clipped to 0.8 | capped |
| − small | 0.9 | not clipped | allowed |

# PPO — Clipped Objective

## Clipped Objective

$$L^{CLIP} = \mathbb{E}_t\Big[\min(r_t(\theta)A_t, clip(r_t, 0.8, 1.2)A_t)\Big]$$

min() picks the more conservative term.

$\epsilon = 0.2 \Rightarrow$ trust region $[0.8, 1.2] \Rightarrow$ stable policy improvement.

## Entropy Bonus

$$H(\pi) = -\sum_a \pi(a|s)\log\pi(a|s)$$

- High entropy $\Rightarrow$ exploration
- Low entropy $\Rightarrow$ policy collapse

$$L_{entropy} = -c_1 H(\pi), \quad c_1 = 0.01$$

# PPO — Critic Loss and Complete Objective

## Critic Loss — Value Estimation

$$L_{\text{critic}} = \frac{1}{T}\sum_t (V(s_t) - G_t)^2$$

$G_t = A_t + V(s_t)$ (target return)

**Interpretation**

- $V(s_t) \rightarrow$ predicted value
- $G_t \rightarrow$ actual return
- Critic learns to minimize prediction error

Weight: $c_2 = 0.5$ (critic supports actor)

## Optimization Tricks

**1. Negative sign on $L^{CLIP}$**

PyTorch minimizes loss, but PPO must maximize policy objective.

$$\text{Optimize } -L^{CLIP}$$

**2. Gradient Clipping**

If gradient norm becomes too large:

$$||\nabla|| > 0.5$$

scale gradients to keep updates stable.

## Complete PPO Loss

$$L_{\text{total}} = -L^{CLIP} + 0.5L_{\text{critic}} - 0.01L_{\text{entropy}}$$

- Actor improves policy
- Critic improves value estimates
- Entropy maintains exploration

# VII : Training Results & Evaluation

# Training Results: $\lambda_2$ vs Training Episodes

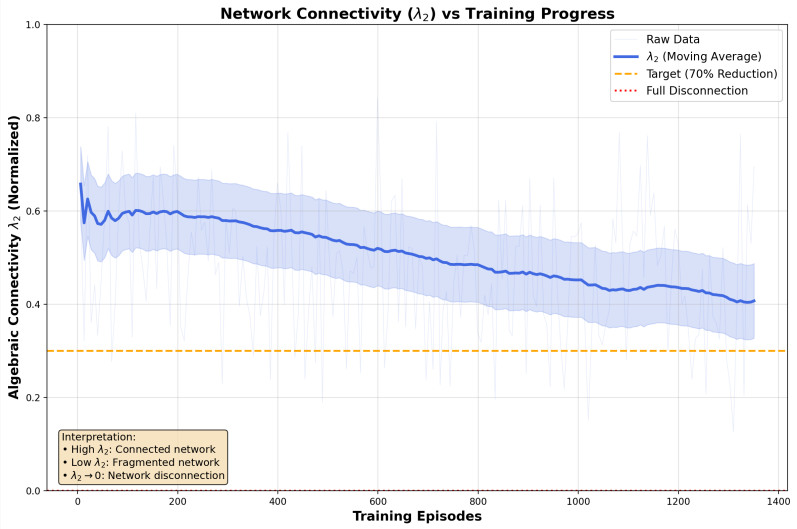



**Figure 1:** $\lambda_2$ vs. Training Episodes

# Training Results: Reward vs Training Episodes

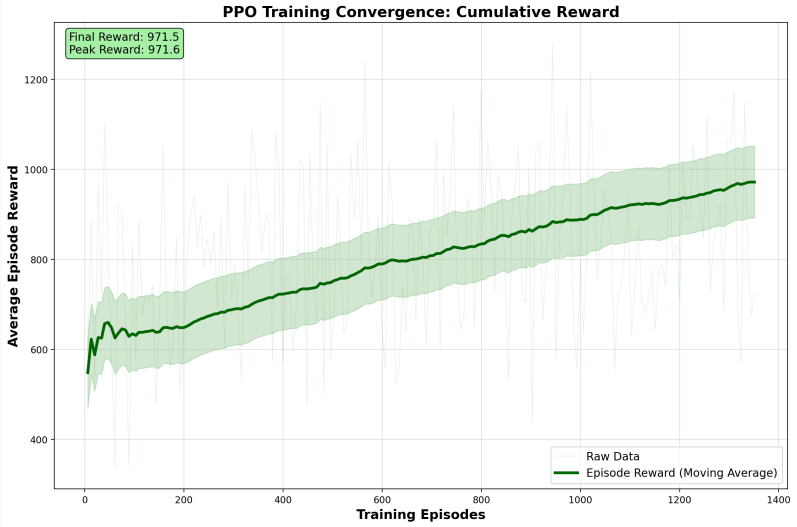



**Figure 2:** Reward vs. Training Episodes

# Training Results: Average Power Comparison

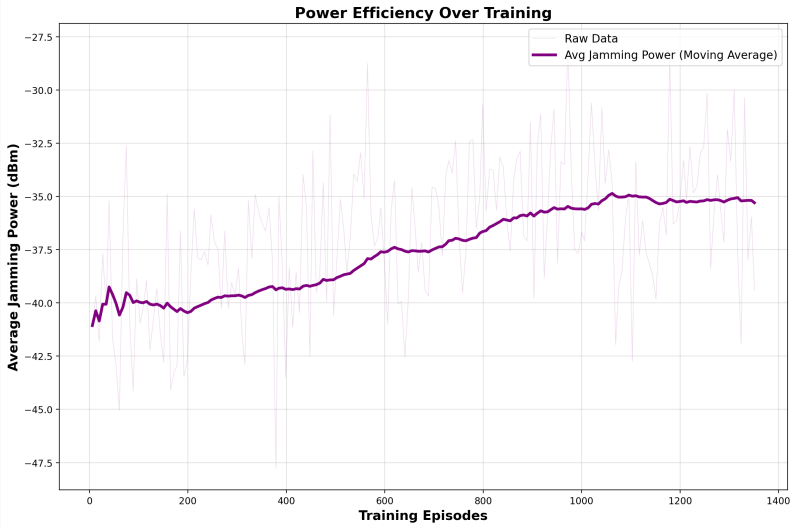



**Figure 3:** Average Power Comparison

# Training Results: Connectivity Before vs After

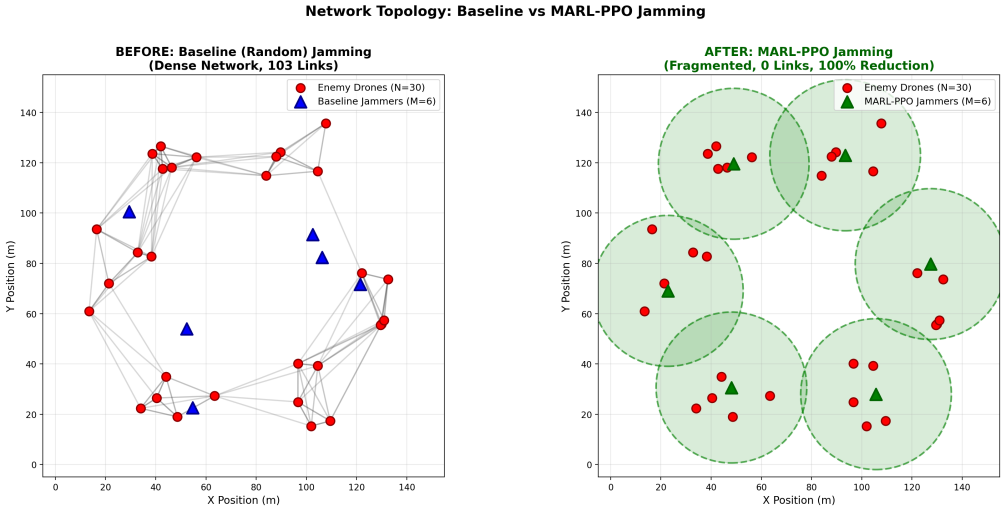



**Figure 4:** Connectivity Before vs. After

# Dataset Evaluation (Military Drone Swarm & Saturation Attack Dataset)

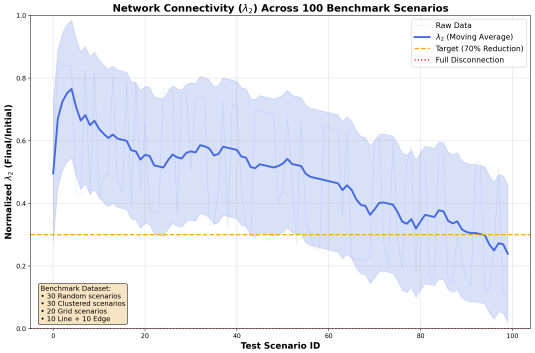



**Figure 5:** $\lambda_2$ vs. Training Episodes

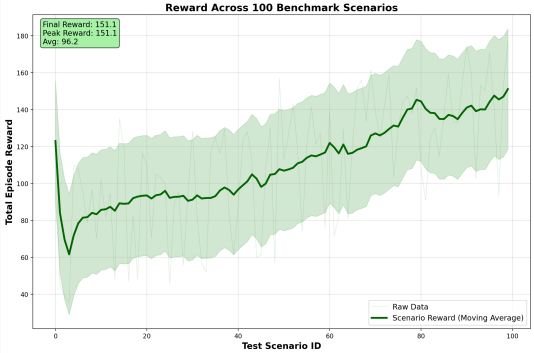



**Figure 6:** Reward vs. Training Episodes

# Thank You

**Questions & Discussion**